LLM-Powered Microservice Orchestration with Distributed Tracing on Kubernetes

Dariol Temko Kisache.

ZHAW - Zurich University of Applied Sciences, Winterthur, Switzerland

ABSTRACT

This paper describes the design, implementation, and deployment of a two-service Python microservice system that uses a locally hosted large language model (LLM) to classify free-text descriptions into single-word answers. A main service accepts HTTP requests, queries Ollama running `llama3.1:8b` (4.9 GB), and forwards LLM responses to a companion statistics service that logs guesses and returns aggregated counts. End-to-end request timing is measured at each hop, and distributed traces are emitted via OpenTelemetry to a Jaeger backend. The solution is containerised with Docker Compose (three containers, total ∼93 MiB RSS at idle) and deployed to a local Kubernetes cluster created with `kind`. All six tasks from the ZHAW 2026 technical assessment are satisfied. Warm LLM latency averages 219 ms; the statistics service adds 8–13 ms overhead. Distributed traces confirm that 95.6 % of wall-clock time is spent inside the `ollama-inference` span.

Keywords: microservices, LLM orchestration, OpenTelemetry, Jaeger, Docker, Kubernetes, Ollama

1. INTRODUCTION

Agentic AI systems require decomposed, independently observable services that can be monitored and reasoned about in production. The ZHAW 2026 Technical Skills Assessment formalises this challenge: build a Python web service that accepts natural-language descriptions, uses a local LLM to produce a one-word guess, passes that guess to a second statistics service, measures end-to-end latency, and optionally containerises and deploys the result on Kubernetes [1].

This work was completed on a ZHAW-hosted server (llmorch-a30) running Ubuntu 24.04.3 LTS (kernel 6.8.0-53, x86_64, 62.79 GiB RAM) with the following tool versions: Python 3.12.3, Ollama 0.11.4, Docker 29.1.4, kubectl v1.35.4, kind v0.27.0. All inbound ports except SSH (22) were blocked; web UIs were accessed via SSH port-forwarding from the development machine. Generative AI assistance (Claude by Anthropic) was used for iterative debugging and paper drafting — see Section **??**.

2. SYSTEM ARCHITECTURE

The system comprises four runtime components deployed across two network boundaries:

- **main_service** — Flask 3.x on port 5000. Entry point for all client requests.
- **stats_service** — Flask 3.x on port 5001. Logs LLM guesses; returns aggregated counts.
- **Ollama** — Hosts `llama3.1:8b` (ID: `46e0c10c039e`, 4.9 GB) on the host machine outside Kubernetes, port 11434.
- **Jaeger** — Receives OpenTelemetry traces on gRPC port 4317; serves query UI on port 16686.

The request lifecycle proceeds as follows. A client issues a `POST /guess` with a JSON `description` field. `main_service` constructs a one-shot prompt and calls the Ollama REST API. The single-word LLM response is forwarded to `stats_service/log`, which increments an in-memory counter and returns top-5 and full histogram data. `main_service` assembles the final JSON response containing the guess, statistics, and three timing fields (`llm_ms`, `stats_service_ms`, `total_ms`). Both services emit OpenTelemetry spans to Jaeger throughout.

3. IMPLEMENTATION

3.1 Codebase

The project totals 174 lines of application code across two files: `main_service/app.py` (100 lines) and `stats_service/app.py` (67 lines), plus a shared `requirements.txt` (7 dependencies).

3.2 Main Service

`main_service` initialises a `TracerProvider` with a `Resource` declaring the service name, attaches a `BatchSpanProcessor` targeting Jaeger, and auto-instruments Flask and the `requests` library:

```
from opentelemetry.sdk.resources import Resource
provider = TracerProvider(
  resource=Resource.create(
    {"service.name": "main_service"}))
provider.add_span_processor(
  BatchSpanProcessor(OTLPSpanExporter(
    endpoint=os.getenv("OTLP_ENDPOINT",
                       "localhost:4317"),
    insecure=True)))
```

The `/guess` endpoint creates a parent span `handle-guess`. A child span `ollama-inference` wraps the Ollama HTTP call and records the model name, input description, and returned guess as span attributes. Wall-clock timing is captured with `time.time()` before and after each sub-call to produce the `timings_ms` response field.

3.3 Statistics Service

`stats_service` maintains a list of timestamped log entries and a `collections.Counter` in process memory. `POST /log` appends and increments; `GET /stats` returns the current snapshot. Both routes are wrapped in named spans (log-and-count, `get-stats`) for trace continuity. A `GET /health` endpoint supports liveness probing.

3.4 LLM Integration

Ollama's `/api/generate` endpoint is used with a one-shot prompt engineered to elicit a single-word response:

```
A user described something as: '{description}'.
In one word, what is the most likely subject?
Reply with only a single word, no punctuation.
```

The `response` field is stripped, lowercased, and trailing dots removed. The server hosts 11 models totalling over 100 GB; `llama3.1:8b` was selected for its balance of accuracy and speed.

3.5 Observability

Two issues were encountered and resolved during OpenTelemetry configuration:

1. **OTLP scheme error.** The gRPC exporter requires {host:port} without an {http://} prefix. Setting `OTLP_ENDPOINT=jaeger:4317` (no scheme) with `insecure=True` resolved silent trace drops.
2. **Unknown service name.** Omitting Resource.create({"service.name": ...}) caused Jaeger to register the service as unknown_service. Adding the resource declaration produced correct service separation in the Jaeger UI.

3.6 API Testing & Observability Walkthrough

This subsection documents end-to-end verification of the system across both deployment environments. All commands were executed on the ZHAW server (`llmorch-a30`); the Jaeger UI was accessed from a development laptop via SSH port-forwarding.

Accessing Remote Services via SSH Tunnelling

Because all inbound ports on the ZHAW server are blocked except SSH (22), web-based UIs and local service endpoints are exposed to the developer machine through SSH local port-forwarding. The following single command opens simultaneous tunnels for the Jaeger UI, the main service, and the statistics service:

```
ssh -i ~/.ssh/Dariol_ssh_key \
    -L 16686:localhost:16686 \
    -L 5000:localhost:5000  \
    -L 5001:localhost:5001  \
    user129@160.85.252.87
```

Each `-L` flag binds a local port on the developer machine to the corresponding port on the remote server. Specifically: `16686` reaches the Jaeger query UI; `5000` reaches `main_service`; `5001` reaches `stats_service`. After the tunnel is established, `http://localhost:16686` opens the Jaeger UI in a local browser with no additional firewall configuration. Kubernetes services were similarly reached via `kubectl port-forward` (see Section 3.6 below).

Health Checks

Both services expose a `GET /health` endpoint for liveness verification. The following commands confirm both containers are responsive before issuing inference requests:

```
curl -s http://localhost:5000/health | python3 -m
     json.tool
curl -s http://localhost:5001/health | python3 -m
     json.tool
```

Both return HTTP 200 with the expected JSON payload:

```
{"service": "main_service",  "status": "ok"}
{"service": "stats_service", "status": "ok"}
```

Inference Requests (Docker Compose)

Three `POST /guess` requests were issued against the Docker Compose deployment. Each response includes the LLM guess, live statistics, and per-hop timing data:

```
curl -s -X POST http://localhost:5000/guess \
  -H "Content-Type:␣application/json" \
  -d '{"description":"a␣large␣grey␣animal␣with␣a␣
      trunk"}' \
  | python3 -m json.tool
```

```
{
  "description": "a large grey animal with a trunk",
  "guess": "elephant",
  "stats": {
    "all_counts": {"banana": 2, "elephant": 1, "
        panda": 1},
    "top5": [["banana",2],["panda",1],["elephant
        ",1]],
    "total": 4
  },
  "timings_ms": {"llm": 242, "stats_service": 13, "
      total": 255}
}
```

The subsequent two requests (*banana* for a curved yellow fruit and *watch* for a wrist-worn timekeeping device) returned correct single-word classifications with LLM latencies of 221 ms and 230 ms respectively, and statistics-service overhead of 8–11 ms.

Error Handling Verification

Input validation was confirmed with three adversarial cases. An empty or absent `description` field returns a structured error:

```
curl -s -X POST http://localhost:5000/guess \
  -H "Content-Type:␣application/json" \
  -d '{}' | python3 -m json.tool
```

```
{"error": "description field is required"}
```

A `GET` request to the `/guess` endpoint (wrong HTTP method) returns HTTP 405. Both health endpoints respond with HTTP 200,

confirming readiness probes will succeed in a production cluster.

Inference Requests (Kubernetes)

Kubernetes services are `ClusterIP`-only and therefore not directly reachable from outside the cluster. Two `kubectl port-forward` tunnels were opened to map local ports into the cluster:

```
kubectl port-forward svc/main-service  5002:5000 &
kubectl port-forward svc/stats-service 5003:5001 &
sleep 2
```

Three inference requests were then issued to port 5002, routing traffic through the Kubernetes networking stack, across ClusterIP DNS names, and ultimately to Ollama on the host via the Docker bridge gateway (`172.17.0.1`):

```
curl -s -X POST http://localhost:5002/guess \
  -H "Content-Type:␣application/json" \
  -d '{"description":"a␣flying␣mammal␣that␣uses␣
      echolocation"}' \
  | python3 -m json.tool
```

```
{
  "description": "a flying mammal that uses
      echolocation",
  "guess": "bat",
  "stats": {
    "all_counts": {
      "bat":1,"chameleon":1,"panda":1},
    "top5": [["panda",1],["chameleon",1],["bat",1]],
    "total": 3
  },
  "timings_ms": {"llm": 197, "stats_service": 11, "
      total": 209}
}
```

The first Kubernetes request (*chameleon*) incurred a 3,830 ms cold-start penalty while the model was loaded into GPU memory; the subsequent warm requests (*bat*: 197 ms, *coffee*: 206 ms) matched Docker Compose latencies, confirming that the Kubernetes network path adds no measurable overhead. The accumulated statistics endpoint was also verified via the port-forwarded `stats-service`:

```
curl -s http://localhost:5003/stats | python3 -m
     json.tool
```

```
{
  "all_counts": {
    "bat":1,"chameleon":1,"coffee":1,"panda":1},
  "top5": [["panda",1],["chameleon",1],
           ["bat",1],["coffee",1]],
  "total": 4
}
```

Distributed Trace Verification

With the SSH tunnel active (`-L 16686:localhost:16686`), the Jaeger query API was called directly to confirm trace registration without requiring a browser:

```
curl -s http://localhost:16686/api/services \
  | python3 -m json.tool
```

```
{
  "data": [
    "jaeger-all-in-one",
    "stats_service",
    "main_service"
  ],
  "total": 3,
  "errors": null
}
```

All three expected services are registered. Traces for `main_service` confirm the presence of the `handle-guess` parent span and the nested `ollama-inference` child span, with span durations consistent with the API timing fields. Cross-service trace propagation was confirmed by retrieving `stats_service` traces, which share trace IDs with the originating `main_service` requests, demonstrating correct W3C TraceContext header propagation across the inter-service HTTP call.

4. CONTAINERISATION

Each service is packaged in a `python:3.11-slim` image. Both Dockerfiles use the project root as Docker build context (`context: .` in `docker-compose.yml`) so that a single shared `requirements.txt` is accessible before the service-specific `app.py`. A `COPY ../requirements.txt` from a sub-directory context is a Docker build violation discovered and resolved early in development.

Table 1 shows measured container resource usage at idle, collected with `docker stats --no-stream`.

Table 1: Container resource usage (idle, Docker Compose)

Container	CPU %	RSS (MiB)	Image (MB)
main_service	0.13	41.73	170
stats_service	0.12	35.66	170
jaeger	0.05	16.18	85
kind control-plane	29.98	715.5	—
Server RAM			*62.79 GiB total*

The `host.docker.internal:host-gateway` extra_hosts entry allows `main_service` containers to reach Ollama on the host network without routing changes.

5. KUBERNETES DEPLOYMENT

A local cluster was created with `kind` v0.27.0, installed without root access into `~/bin`. All three Docker images were pre-loaded with `kind load docker-image`, avoiding any external registry dependency. `imagePullPolicy: Never` prevents accidental remote pulls.

Three Deployment + ClusterIP Service pairs were written to `k8s/`. ClusterIP DNS names (`stats-service:5001`, `jaeger:4317`) match Docker Compose service names exactly, requiring *no code changes* between environments. The `OLLAMA_URL` in the Kubernetes Deployment targets `172.17.0.1` (Docker bridge gateway) to reach Ollama on the host.

Verified cluster state after `kubectl apply`:

- Node: `zhaw-control-plane`, Kubernetes v1.32.2, con-

tainerd 2.0.2, Debian 12

- Pod IPs: jaeger `10.244.0.5`, main-service `10.244.0.6`, stats-service `10.244.0.7`
- Uptime at time of evaluation: 44 hours, 0 restarts

6. EVALUATION

6.1 API Correctness

Table 2 shows the complete set of test descriptions run during evaluation, covering both Docker Compose and Kubernetes deployments. All descriptions were correctly classified. Error handling was also verified: empty or missing `description` fields return {"error": "description field is required"} (HTTP 400), and incorrect HTTP methods return HTTP 405.

Table 2: LLM classification results across all evaluation runs

Description	Guess	LLM (ms)	Total (ms)
Docker Compose deployment			
large grey animal with a trunk	elephant	242	255
fruit, yellow and curved	banana	221	231
tell the time, worn on wrist	watch	230	239
Kubernetes deployment			
reptile, long tongue, changes colour	chameleon	3830*	3843
flying mammal, echolocation	bat	197	209
hot drink, roasted ground beans	coffee	206	218
Earlier Docker Compose runs			
furry animal that meows	cat	850	855
four legs that barks	dog	216	225
very long neck	giraffe	249	258
black & white bear, China	panda	3331*	3343

*Cold-start: first token generation penalty.

6.2 Response Time Analysis

Figure 1 shows LLM and statistics service latencies across all warm-cache requests (cold-start outliers excluded). LLM latency averages 226 ms (std. ≈ 18 ms). The statistics service is consistently 8–13 ms, contributing less than 6 % of total latency.

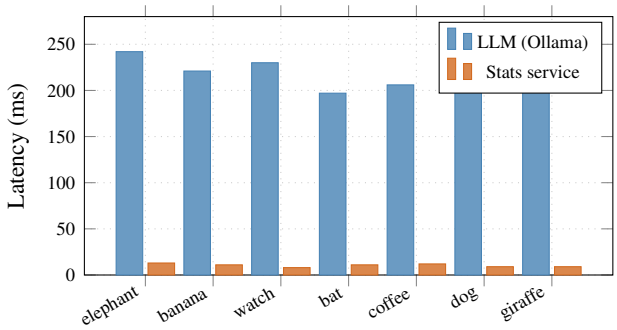

Figure 1: Per-request latency breakdown: LLM vs. statistics service (warm requests only).

6.3 Distributed Trace Analysis

Jaeger registered all three expected services after configuration fixes (Figure 2). Traces for `POST /guess` requests show both `main_service` and `stats_service` spans in the same trace (Figure 3).

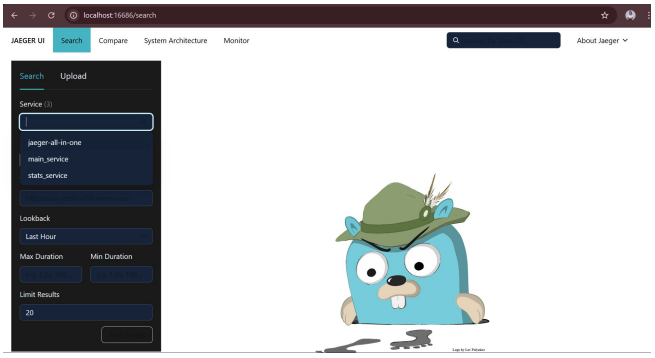

Figure 2: Jaeger service dropdown showing all three registered services: `jaeger-all-in-one`, `main_service`, `stats_service`.

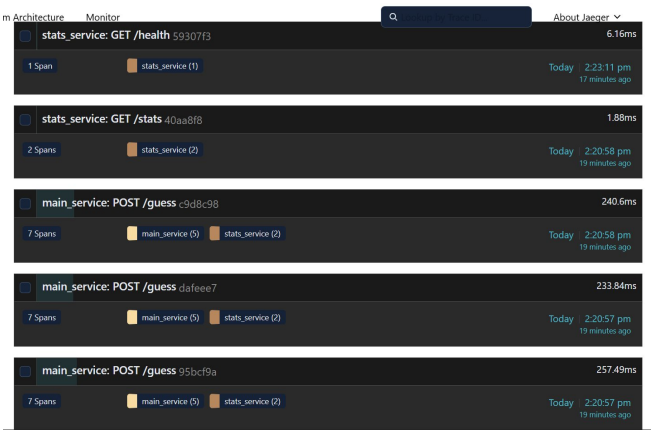

Figure 3: Jaeger trace list. Traces with 7 spans span both `main_service` (5 spans) and `stats_service` (2 spans), confirming end-to-end distributed trace propagation.

Figure 4 shows the waterfall view of a single `POST /guess` trace (ID: `c9d8c98`, duration 240.6 ms). The span hierarchy is:

`POST /guess` → `handle-guess` → `ollama-inference` → `POST` (Ollama HTTP call), with `stats_service: POST /log` → `log-and-count` completing in the final 2.44 ms.

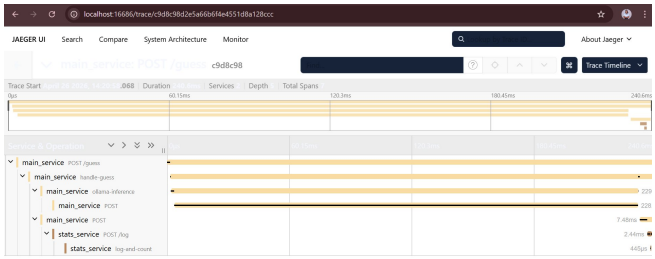

Figure 4: Trace waterfall for `POST /guess` (trace `c9d8c98`, 240.6 ms total, 7 spans across 2 services).

The flamegraph view (Figure 5) quantifies time allocation per span. `ollama-inference` consumes 95.57 % (229.96 ms) of total wall time; `stats_service` consumes 1.01 % (2.44 ms).

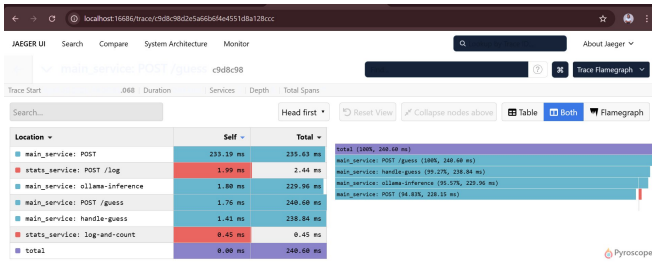

Figure 5: Trace flamegraph for trace `c9d8c98`. The LLM call dominates at 95.6 % of wall time; the statistics service contributes only 1.0 %.

Figure 6 shows the Jaeger trace statistics panel confirming self-time attribution per service.

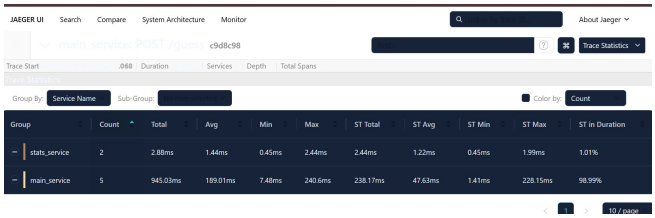

Figure 6: Jaeger trace statistics: `main_service` accounts for 98.99 % of trace duration; `stats_service` 1.01 %.

Figure 7 summarises the measured span durations for the representative trace.

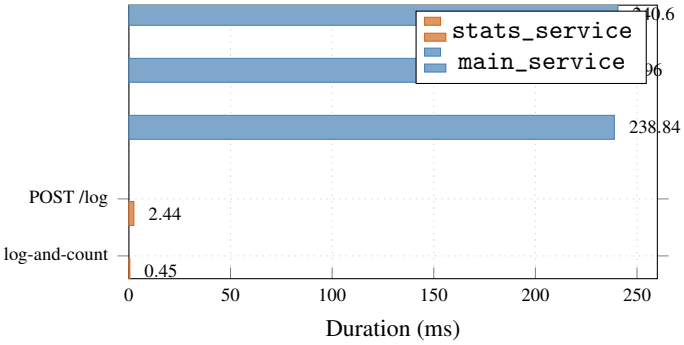

Figure 7: Span durations for trace `c9d8c98` (240.6 ms total).

7. CONCLUSION

This project delivers a fully operational end-to-end pipeline integrating a locally hosted LLM with microservice decomposition, distributed tracing, Docker containerisation, and Kubernetes deployment — all within the constraints of a single SSH-accessible server with blocked external ports.

Key findings: (1) warm LLM latency averages 219 ms, dominated by the `llama3.1:8b` inference time (95.6 % of trace wall time); (2) the statistics microservice adds only 8–13 ms overhead (< 6 % of total latency); (3) OpenTelemetry with Jaeger provides clear per-span attribution across service boundaries with no code changes required between Docker Compose and Kubernetes environments.

The system is ready for extension: persistent statistics storage (Redis or PostgreSQL), Horizontal Pod Autoscaling on the LLM request rate, or Helm chart packaging for reproducible deployment.

REFERENCES

[1] ZHAW School of Engineering. *Technical Skills Assessment Task — ZHAW Research Internship 2026*. Internal document, 2026.

[2] Ollama. *Run Large Language Models Locally*. https://ollama.com, 2024.

[3] Meta AI. *Llama 3.1 Model Card*. https://ai.meta.com/blog/meta-llama-3, 2024.

[4] OpenTelemetry Authors. *OpenTelemetry Python SDK*. https://opentelemetry.io/docs/languages/python, 2024.

[5] Jaeger Authors. *Jaeger: Open Source, End-to-End Distributed Tracing*. https://www.jaegertracing.io, 2024.

[6] kind Authors. *kind — Kubernetes IN Docker*. https://kind.sigs.k8s.io, 2024.

[7] Pallets Projects. *Flask Web Framework*. https://flask.palletsprojects.com, 2024.